3. ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ СЕРВЕРНОЙ ЧАСТИ

3.1 Разработка схемы взаимодействия микросервисов

Проектирование серверной части приложения такси начинается с определения того, из каких компонентов она будет состоять и как эти компоненты будут между собой общаться. Исходя из результатов анализа, проведённого в предыдущих главах, было решено строить систему на основе микросервисной архитектуры. Логика тут простая: каждый крупный блок функциональности выносится в отдельный сервис, у каждого своя БД, и они общаются по сети. Это даёт гибкость в плане масштабирования и независимого развития отдельных частей системы [1].

В разрабатываемой системе можно выделить два типа сервисов: инфраструктурные и бизнесовые. К инфраструктурным относятся Eureka (для поиска и обнаружения сервисов), Config Server (хранилище общих настроек) и API Gateway (единая точка входа). Бизнесовых сервисов пять: сервис аутентификации, сервис пассажиров, сервис водителей, сервис поездок и сервис рейтинга. Каждый из них отвечает за свою область и имеет собственную БД. Схему того, как это всё связано, можно увидеть на рисунке 3.1.

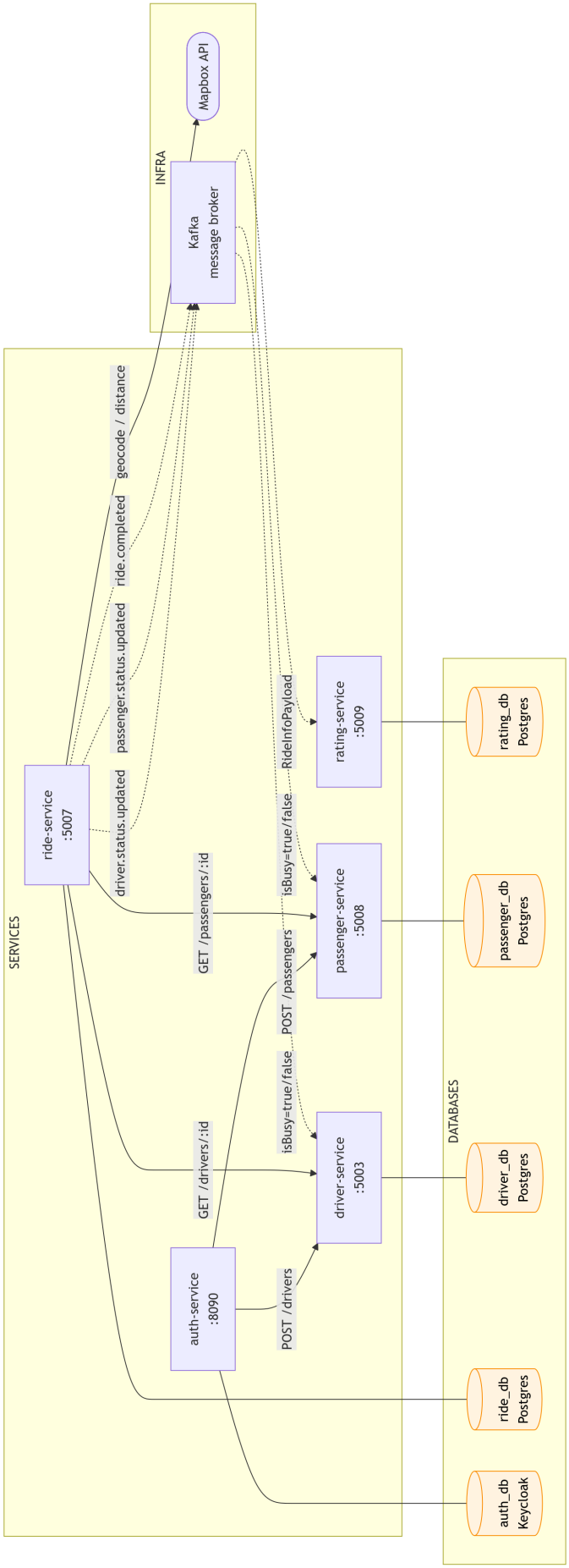

Рисунок 3.1 – Схема взаимодействия микросервисов

API Gateway работает на порту 8080. Он принимает все входящие запросы от клиентов, проверяет цифровые подписи и направляет запросы к нужным сервисам. Например, запросы на /api/v1/passengers/** идут в сервис пассажиров, на /api/v1/drivers/** - в сервис водителей, на /api/v1/rides/** - в сервис поездок. Все сервисы регистрируются в Eureka, поэтому gateway может обращаться к ним по именам, а система сама находит нужный сервис и распределяет нагрузку. Config-server, в свою очередь, хранит общие настройки для всех сервисов, и при старте каждый сервис получает конфигурацию оттуда.

Теперь про то, как сервисы общаются между собой. Тут используется два подхода: синхронный через HTTP-вызовы и асинхронный через систему обмена сообщениями. Синхронные вызовы применяются там, где нужно получить ответ прямо сейчас. Например, когда сервис аутентификации регистрирует нового пользователя, он сначала создаёт учётную запись, а потом через HTTP-запрос обращается к сервису пассажиров или сервису водителей, чтобы создать профиль. Другой пример: сервис поездок перед назначением водителя на заказ проверяет, что водитель существует и не занят. Все эти вызовы защищены: каждый сервис авторизуется как доверенное приложение, а не как конкретный пользователь.

Чтобы синхронные вызовы не ломали всю систему, если какой-то сервис временно недоступен, используются механизмы обработки ошибок: делается несколько повторных попыток, а при постоянных сбоях вызовы временно прекращаются, чтобы один упавший сервис не повалил всю систему [1].

Асинхронное взаимодействие используется для событий, где не нужно ждать ответа прямо сейчас. Например, при создании поездки меняется статус пассажира на ”занят при назначении водителя - статус водителя. Или когда поездка завершена - нужно сказать сервису рейтинга, что можно выставлять оценки. Для таких вещей сервис поездок отправляет сообщения через систему обмена сообщениями, а сервисы пассажиров, водителей и рейтинга получают эти сообщения и обрабатывают их.

Но тут есть нюанс: если отправить сообщение, а потом что-то пойдёт не так, данные могут разойтись. Чтобы такого не было, используется следующий подход: сначала данные сохраняются в БД вместе с изменением статуса поездки, а потом уже отправляются получателю. Если отправка не удалась - данные остаются в БД и будут отправлены при следующей попытке. Так гарантируется, что сообщение дойдёт хотя бы один раз [8].

Отдельно стоит сказать про авторизацию и аутентификацию. За это отвечает внешняя система управления доступом. Когда пользователь регистрируется или заходит через сервис аутентификации, тот связывается с этой системой, получает цифровой ключ доступа и передаёт его клиенту. Дальше клиент при каждом запросе прикладывает этот ключ. API Gateway проверяет ключ, достаёт оттуда идентификатор пользователя и передаёт его другим сервисам в специальном заголовке. Таким образом сервисы могут реализовать адрес /me, чтобы получить профиль текущего пользователя.

3.2 Проектирование структуры базы данных и геоинформационных индексов

Для каждого микросервиса в системе используется своя собственная БД. Такой подход гарантирует, что сервисы слабо связаны: каждый владеет своими данными, и никто не лезет в чужую БД напрямую [1]. Изменения схемы отслеживаются с помощью специального инструмента, что позволяет контролировать все изменения и при необходимости

откатываться назад.

В сервисе водителей есть две основные таблицы: driver и car. В таблице driver хранятся ФИО, адрес электронной почты, телефон, пол, отметки о том, активен ли пользователь и занят ли он, а также привязка к системе управления доступом. В таблице car - цвет, марка, номер машины и ссылка на таблицу водителей. У одного водителя может быть несколько машин. В сервисе пассажиров всё проще - одна таблица passenger с почти таким же набором полей. В сервисе поездок хранятся сами поездки, адреса и сообщения для отправки, а в сервисе рейтинга - информация о завершённых поездках и оценки.

Центральная таблица всей системы - это таблица поездок в сервисе поездок. В ней содержатся идентификаторы пассажира и водителя, адреса отправления и назначения (через ссылки на таблицу адресов), статус поездки (12 возможных состояний), цена, дистанция, время создания, подачи и завершения. Статусы хранятся в виде целого числа и преобразуются с помощью механизма преобразования типов. Важный момент: идентификаторы водителя и пассажира в этой таблице - это условные ссылки, то есть связь не на уровне СУБД, а на уровне бизнес-логики, потому что физически водители и пассажиры хранятся в других базах данных.

Таблица адресов заслуживает отдельного внимания, потому что она отвечает за геоинформацию. В ней хранятся: название адреса строкой и координаты (широта, долгота) в виде чисел с плавающей точкой. Изначально планировалось использовать геоинформационные расширения баз данных для поиска водителей рядом с пассажиром. Но в текущей версии MVP эта задача решается иначе: вся работа с геоданными вынесена на внешний сервис. Когда пользователь вводит адрес, сервис поездок отправляет запрос в картографический сервис, получает координаты, а потом считает дистанцию между точками.

Чтобы не обращаться к картографическому сервису каждый раз, результаты сохраняются в кэш. Причём кэширование трёхуровневое: адреса целиком (название + координаты), результаты преобразования адресов в координаты и дистанции между парами адресов. Таким образом экономятся и время, и ресурсы.

Ещё одна важная таблица - таблица сообщений в сервисе поездок. Это основа подхода, о котором говорилось ранее. В ней сохраняются: название канала, ключ сообщения, само сообщение в текстовом виде, временные метки создания и отправки, а также отметка о том, отправлено ли сообщение. Отдельный указатель позволяет быстро находить неотправленные сообщения. Специальная задача каждую секунду проверяет эту таблицу и отправляет всё, что ещё не ушло.

Поскольку сервисов несколько, и каждый может быть запущен в нескольких экземплярах, задача отправки должна работать только на одном из них, чтобы не было дублирования. Для этого используется механизм блокировок через отдельную таблицу в той же БД. Раз в сутки запускается задача очистки, которая удаляет уже отправленные сообщения, чтобы таблица не разрасталась.

3.3 Разработка API и протоколов обмена данными

API серверной части спроектирован в REST-стиле. Все запросы приходят на единый шлюз на порту 8080, а оттуда уже направляются к конкретным сервисам. Формат данных - JSON. Общая схема API показана в таблице 3.1.

Таблица 3.1 – Маршрутизация API Gateway

Путь	Сервис	Авторизация	Описание
/auth/**	аутентификация	нет	Регистрация и вход
/passengers/**	пассажиры	да	Работа с пассажирами
/drivers/**	водители	да	Работа с водителями
/cars/**	водители	да	Работа с машинами
/rides/**	поездки	да	Управление поездками
/ratings/**	рейтинг	да	Оценки и рейтинг

Сервис аутентификации отвечает за вход и регистрацию. У него есть открытые адреса: POST /signup (создание пользователя и его профиля), POST /signin (получение ключа доступа), POST /refresh (обновление ключа) и POST /logout (отзыв старого ключа). После регистрации сервис аутентификации через HTTP-запрос обращается к сервису пассажиров или сервису водителей, чтобы создать профиль.

Сервисы пассажиров и водителей предоставляют операции создания, чтения, обновления и удаления для своих записей. Помимо обычных адресов для работы с конкретными записями у них есть адрес /me, который возвращает профиль текущего пользователя по его идентификатору, полученному из специального заголовка. Это удобно для клиента - не нужно хранить внутренние идентификаторы. Также есть список всех записей с возможностью постраничного просмотра.

Самый интересный сервис с точки зрения API - это сервис поездок. Для начала поездки клиент отправляет POST /rides с указанием адреса отправления, назначения и своего ID. Система проверяет, что пассажир не занят, преобразует адреса в координаты через картографический сервис, считает дистанцию и цену (10 условных единиц за километр), создаёт поездку и сразу переводит её в статус ожидания водителя, отправляя сообщение о том, что пассажир занят.

Дальше поездка живёт по конечному автомату, который включает 12 возможных состояний и переходы между ними. Вот как выглядит полный жизненный цикл поездки: CREATED - WAITING_FOR_DRIVER - DRIVER_ASSIGNED - ACCEPTED - ON_THE_WAY_TO_PASSENGER - PASSENGER_PICKED_UP - ON_THE_WAY - COMPLETED - PAYMENT_PENDING - PAID - RATE. Из многих состояний можно отменить поездку (CANCELLED). Визуально диаграмма состояний показана на рисунке 3.2.

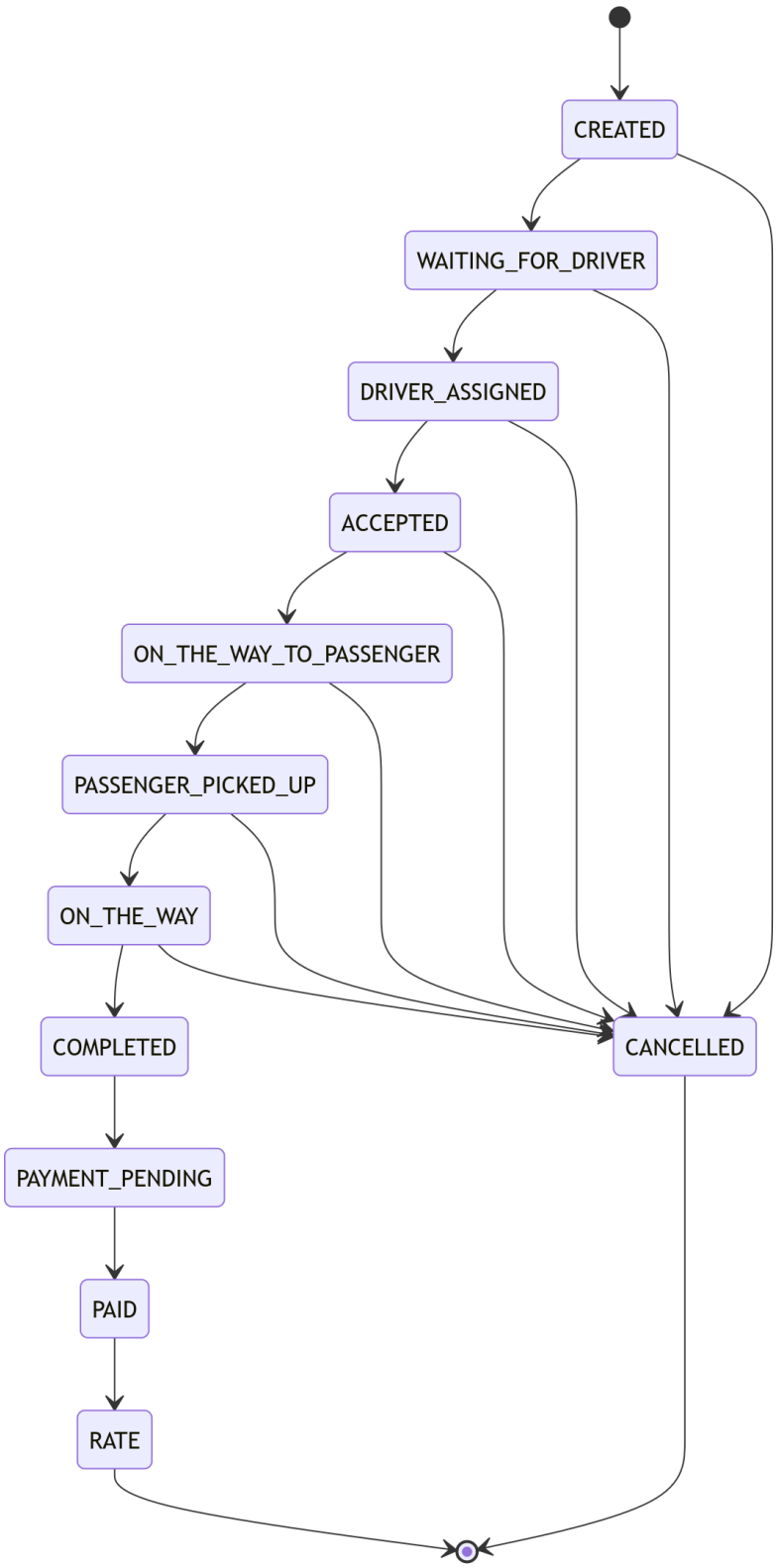

Рисунок 3.2 – Диаграмма состояний поездки

Переходы между состояниями строго контролируются: система проверяет допустимость перехода из текущего статуса в новый. Например, из CREATED разрешены только переходы в WAITING_FOR_DRIVER или CANCELLED, а из COMPLETED - только в PAYMENT_PENDING. При определённых переходах возникают побочные эффекты: при переходе в PASSENGER_PICKED_UP фиксируется время подачи, при переходе в COMPLETED - время завершения поездки, а также публикуются события освобождения

водителя и пассажира в Kafka.

Сервис рейтинга работает проще всех. Его основная задача - принимать оценки после завершённых поездок. Сервис рейтинга получает сообщение о завершении поездки и создаёт запись, фиксируя этот факт. После этого пассажир или водитель могут отправить POST /ratings с оценкой (от 0 до 5), комментарием и указанием, кто оценивает. Средний рейтинг считается программно: берутся последние N оценок (по умолчанию 20) и вычисляется среднее арифметическое. Это позволяет гибко настраивать количество учитываемых оценок.

Отдельно стоит сказать про каналы передачи сообщений, которые соединяют сервисы в единую систему. Всего их три: один меняет отметку ”занят”у пассажира, второй - аналогично для водителя, третий передаёт информацию о завершённой поездке в сервис рейтинга. Каждый канал настроен так, чтобы сообщения не терялись и обрабатывались без дублирования.